

MỤC LỤC

MỤC LỤC.....	1
Phần I. Lý thuyết:.....	2
1. Tìm hiểu các khái niệm.....	2
a. Authentication (Xác thực) ?.....	2
b. Authorization (Phân quyền) ?.....	2
2. Phân biệt cơ chế xác thực.....	3
a. Session-based.....	3
b. Token-based.....	3
3. Tìm hiểu JWT (JSON Web Token).....	4
a. JWT là gì ?.....	4
b. Lợi ích của JWT ?.....	4
4. Cấu trúc của JWT ?.....	5
Phần II. Thực hành.....	7

Tuần 3: Authentication & Authorization

Phần I. Lý thuyết:

1. Tìm hiểu các khái niệm

a. Authentication (Xác thực) ?

Authentication là cơ chế gán định danh cho request gửi đến server - ví dụ xác định người gửi request là ai dựa vào *username/password* hoặc *token*.

Django REST Framework (DRF) dùng các lớp *authentication classes* để thực hiện điều này, và nếu xác thực thành công, **request.user** sẽ là một đối tượng **User**, còn **request.auth** lưu thông tin bổ sung như token.

Lưu ý: Xác thực chỉ định danh chứ không quyết định có cho phép truy cập hay không.

b. Authorization (Phân quyền) ?

Authorization diễn ra sau xác thực, dùng để kiểm tra xem người dùng có được phép truy cập tài nguyên hay thực hiện hành động cụ thể không. Trong DRF, điều này do các *permission classes* xử lý, ví dụ **IsAuthenticated** hoặc **IsAdmin**.

2. Phân biệt cơ chế xác thực

a. Session-based

- Khi người dùng đăng nhập, server tạo **session** và lưu trong database.
- Server trả session ID cho trình duyệt giữ session ID và lưu trong cookie.
- Mỗi request sau đó gửi cookie lên để server đối chiếu session ID để xác định người dùng.
- Phù hợp cho web truyền thống (server-rendered), nhưng khó mở rộng khi có nhiều client (mobile, SPA) vì tốn không gian lưu trữ của server.

b. Token-based

- Khi đăng nhập người dùng đổi username-password để lấy **token** (ví dụ JWT) từ server.
- Client lưu token (thường trong localStorage hoặc header).
- Mỗi request gửi token qua header **Authorization: Bearer <token>**.
- Server không cần lưu trạng thái - dễ mở rộng, phù hợp API REST.

3. Tìm hiểu JWT (JSON Web Token)

a. JWT là gì ?

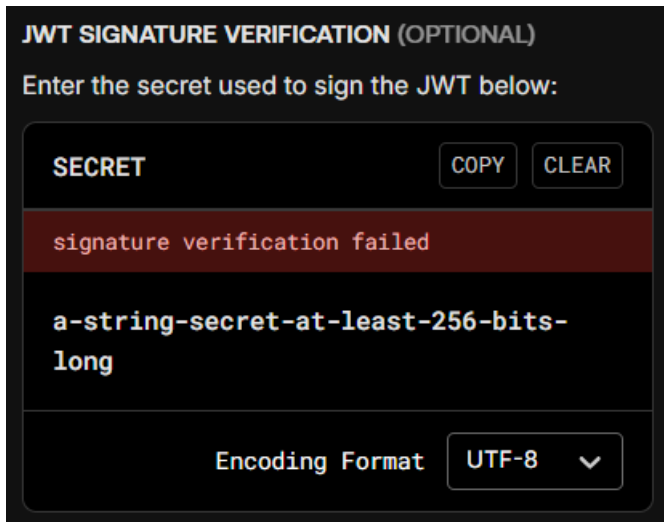
JWT (JSON Web Token) là một **chuẩn mã hóa** để truyền thông tin giữa client và server dưới dạng một chuỗi **token** có thể xác minh được.

Token gồm 3 phần: **Header.Payload.Signature**, được mã hóa bằng Base64 và ký bằng thuật toán như HMAC hoặc RSA để đảm bảo không bị sửa.

b. Lợi ích của JWT ?

- **Stateless:** Server không cần lưu trạng thái đăng nhập, giúp dễ mở rộng hệ thống.
- **Nhanh:** Xác thực chỉ cần kiểm tra chữ ký token, không truy cập database.
- **Dễ tích hợp:** Dùng tốt với nhiều loại client (web, mobile, microservice).
- **Tùy biến:** Có thể nhúng thông tin như **user_id, role, exp** ngay trong payload.
- **Bảo mật:** Nếu dùng HTTPS + secret key mạnh, rất khó giả mạo.

JWT thường dùng trong các API hiện đại để thay thế session, đặc biệt với SPA hoặc mobile app.



Phần II. Thực hành

Phát triển từ Project CRUD Tuần 2

- ☒ 1. Tạo API đăng nhập trả về JWT
- ☒ 2. Tạo API kiểm tra JWT
- ☒ 3. Hashing Mật khẩu
- ☒ 4. Tạo Middleware/Filter Xác thực
- ☐ 5. Phân quyền truy cập dựa trên role trong JWT (**Em sẽ bổ sung sau ạ**)
- ☒ 6. Refresh Token

<https://youtu.be/RTGYLfmtbhw>

Output

- Trình bày lý thuyết.
- Làm đầy đủ phần thực hành. Chụp/quay màn hình các kết quả.